

Claude for STEM Professors

From zero to a deployed course app: the accounts, the tokens, and three projects you can put in front of students.

So, a colleague types a sentence, an app appears, and everyone claps. I have given that demo. What the demo skips is the hour of account setup that nobody enjoys and everybody survives, which is most of what this guide is. You pay that hour once.

You do not need a programming background. If you can follow a recipe and copy-paste, you can do all of this.

Version: 21 Aug 2026. I checked every click-path against the vendor docs in August 2026. Buttons wander. If something is not where I say it is, have a look at the linked docs and trust those instead. Later versions carry a later date up here so you can tell which copy you are holding.

Prefer paper? The PDF of this version is in the repo: `claude-for-stem-professors-2026-08-21.pdf`. Links print with their URLs spelled out so it still works away from a screen. Run `./build-pdf.sh` after edits to rebuild it.

The three destination projects (details in Part 6):

1. **A course website**, built and deployed to Netlify in under an hour
2. **A practice-problem app** for your subject, auto-graded, students use it before exams
3. **A Canvas assistant** that pulls your roster and assignment data and drafts announcements, running on Render

What you need

- A computer with a web browser. Parts 1 through 6 need nothing else.
- About an hour for setup, then an afternoon per project.
- Money: every service here has a free tier that is enough for these projects. Claude Pro (\$20/month) is the one upgrade to consider if you will use this weekly.
- Coffee. I do this with a cortado at 6am, but you do you.

Contents

- Part 1: Set up Claude
 - Part 2: Create your accounts
 - Part 3: Get your tokens
 - Part 4: Connect Claude to your accounts
 - Part 5: Token hygiene
 - Part 6: Three projects
 - Part 7: Scheduled jobs with GitHub Actions
 - Part 8: Appendix, Claude Code
 - Troubleshooting
-

Part 1: Set up Claude

1.1 Create a Claude account

1. Go to claude.ai (<https://claude.ai>).
2. Click **Continue with Google** and pick your Google account.
3. Done. You are on the Free plan.

1.2 Free vs Pro

Free is a real working tier. The difference is mostly capacity.

	Free (\$0)	Pro (\$20/month, \$17 if annual)
Usage	Limited budget that resets every 5 hours	At least 5x Free, plus a weekly cap
Chat, web search, file uploads	Yes	Yes
Projects, Artifacts, file creation	Yes	Yes
Connectors (Part 4)	Directory connectors, plus one custom connector	Directory plus custom connectors
Model choice	Current default model	Full model picker, including the strongest models
Claude Code (Part 7)	No	Yes
Advanced Research	No	Yes

I suggest starting on Free, doing Project 1, and upgrading the first time the limit stops you mid-thought. For some people that is week one. For some it never happens. This table also goes stale faster than anything else in the guide, so claude.com/pricing (<https://claude.com/pricing>) wins any argument with it.

If you do go Pro, my referral link (<https://claude.ai/referral/X0jtHxAOEA>) throws a small thank-you my way. Feel free to ignore it. The guide is the same either way.

University note. Some universities have institutional Claude for Education agreements with different data protections than a personal account. If yours does, use it, especially for anything touching Canvas (Part 3.5 explains why). Ask your IT or teaching-technology office.

1.3 Turn on network egress (do this in a browser)

One setting is easy to miss and makes a real difference. In Claude's settings you will find this:

Allow network egress

Give Claude network access to install packages and libraries in order to perform advanced data analysis, custom visualizations, and specialized file processing. Monitor chats closely as this comes with security risks.

Without it, Claude's code environment is sealed off and cannot install the libraries that most data work needs. Turn it on and it can pull down a plotting library, a PDF parser, a statistics package, whatever the job needs. For the projects here, turn it on.

Where: `claude.ai` (<https://claude.ai>) in a web browser, then **Settings**, then **Capabilities**, then scroll to the **very bottom** of the page. It is the last item.

The catch: the phone app does not show this setting at all. You have to do it once in a browser on a laptop or desktop. It then applies to your whole account, so your phone gets it too.

That warning at the end is not boilerplate. Network access means the code Claude runs for you can reach the internet, and two things follow. It installs software written by strangers, which is the same bet you make every time you install anything. And a token you pasted into that chat could in principle be sent somewhere. This is why Part 5 nags about rotating tokens. Have a look at what Claude proposes before you approve it, the way you would skim a script a student emailed you.

Part 2: Create your accounts

Three accounts, all free, all reachable with the **Continue with Google** button. Ten minutes total.

Which Google account? Your UW account works fine for all three services, and it is the simpler answer: one login you already use daily, and your .edu address qualifies you for GitHub's free education benefits. Use it.

Faculty do not change institutions often, so the "what if I leave" worry is smaller than it looks. If it does happen, GitHub lets you add a second email and switch your primary address without losing a single repo. Five minutes, on a day when you will have bigger things to sort out.

Two exceptions. Use a personal account if the work is personal (consulting, a book, anything you would rather not explain to a chair), or if your campus IT blocks third-party sign-ins, which some units do.

2.1 GitHub

GitHub stores your code and its full history. Every project in this guide lives in a GitHub repository.

1. Go to github.com/signup (<https://github.com/signup>).
2. Click **Continue with Google** and choose your account. (Google login for GitHub has existed since mid-2025; if you last looked before that, it is new.)
3. Pick a username. It is public and hard to change, so choose something you would put on a syllabus.
4. Complete the email or device verification code GitHub sends you.

Two things to do now, while you are in there. Both live in github.com/settings/security (<https://github.com/settings/security>):

- **Set a password or passkey.** Accounts created through Google have no password, which means no way in if your Google account is ever locked.
- **Turn on two-factor authentication.** GitHub will nag you about this anyway.

2.2 Netlify

Netlify hosts websites and frontend apps: your course site, your quiz app. The free tier is generous and fine for a class.

1. Go to app.netlify.com/signup (<https://app.netlify.com/signup>).

2. Click **Sign up with Google**. (GitHub, GitLab, Bitbucket, and email also work. Signing up with GitHub saves one password and makes repo linking slightly smoother later, but Google is fine.)
3. Answer or skip the onboarding questions. You land on an empty dashboard. Good.

2.3 Render

Render hosts backends: apps with a server, a database, or a secret they must keep (like your Canvas token in Project 3). Free-tier services sleep when idle and take a moment to wake up, which is fine for a personal tool. Details at render.com/docs/free (<https://render.com/docs/free>).

1. Go to dashboard.render.com/register (<https://dashboard.render.com/register>).
 2. Click the **Google** button. (GitHub, GitLab, and email also work.)
 3. Confirm your email if asked. You land on an empty dashboard.
-

Part 3: Get your tokens

A token is a password with a narrower job. It lets a program act as you on one service without ever seeing your real password. The pattern is always the same: generate a token, hand it to Claude in a chat, let Claude do the work, then delete or rotate it. Treat a token like a password. That is what it is.

Every token below is shown to you **exactly once**, at the moment of creation. Copy it somewhere safe (a password manager is the right place) before closing the page.

3.1 GitHub classic token

This is the workhorse. With it, Claude can create repositories, push code, and set up deploy pipelines on your behalf.

1. On github.com (<https://github.com>), click your profile photo (top right), then **Settings**.

2. In the left sidebar, scroll to the bottom and click **Developer settings**.
3. Click **Personal access tokens**, then **Tokens (classic)**. Direct link: github.com/settings/tokens (<https://github.com/settings/tokens>).
4. Click **Generate new token**, then **Generate new token (classic)**. GitHub may ask you to re-authenticate.
5. **Note**: name it for its job, e.g. `claude-course-projects`.
6. **Expiration**: 90 days. Do not pick "no expiration"; future-you will forget this token exists.
7. **Scopes**: check the entire **repo** box, and check **workflow**. The first lets Claude manage repositories; the second lets it set up automated deploys. Leave everything else unchecked.
8. Click **Generate token** and copy the token (it starts with `ghp_`). This is the only time you will see it.

When the token expires, generating a new one takes two minutes and the old prompts still work.

3.2 Netlify personal access token

You may not need this one. The Netlify connector in Part 4.2 uses a browser sign-in instead of a token, and that is the better path. Get a token only if you want the paste-a-token route or if a project asks for one.

1. On app.netlify.com (<https://app.netlify.com>), click your avatar, then **User settings**.
2. Click **Applications**, then under **Personal access tokens** click **New access token**. Direct link: app.netlify.com/user/applications (<https://app.netlify.com/user/applications>).
3. Name it, set an **expiration date**, click **Generate token**, copy it once.

One quirk: reset your Netlify password and every existing token dies with it.

3.3 Render API key

1. On dashboard.render.com (<https://dashboard.render.com>), open your **Account Settings** and find the **API Keys** section. Direct link: dashboard.render.com/u/settings (<https://dashboard.render.com/u/settings>).
2. Click **Create API Key**, name it, copy it once.

Note: a Render API key has broad access to your whole account, with no way to narrow it. Guard it accordingly, and prefer the browser sign-in route in Part 4.3 when you can.

3.4 Google OAuth client (for app logins)

The other credentials here let Claude act as you. This one is different: it lets *other people* prove who they are to an app you built. Project 3 uses it so your Canvas dashboard asks for a Google sign-in instead of sitting behind one shared password.

Google reorganized this console during 2025 and most guides on the web still describe the old menus. What follows is the current layout.

1. Go to console.cloud.google.com (<https://console.cloud.google.com>) and sign in. Create a project (top-left project selector, then **New project**). Name it for the app, e.g. `canvas-dashboard`. Switch to it before continuing, which is the step everyone forgets.
2. In the left menu, open **Google Auth Platform**. On a fresh project it offers a **Get started** wizard. Fill in an app name and your support email.
3. **Audience** is the one real decision. **Internal** limits sign-in to accounts in your own Google Workspace organization, which is what you want if your university runs on Google and your project sits inside its organization. If Internal is greyed out, your project is not under the org, so pick **External**. External starts in testing mode, where only the test users you list can sign in, and that is fine for a dashboard with one user.
4. Under **Data access**, add only `openid`, `email`, and `profile`. These are non-sensitive scopes, so Google will not put you through app verification. Ask for anything more and you inherit a review process you do not want.

5. If you chose External, add yourself under **Test users**.
6. Open **Clients**, click **Create client**, choose **Web application** as the type. Anything else gives you the wrong OAuth flow.
7. Under **Authorized redirect URIs**, add both of these, adjusting the hostname:
 - `http://localhost:5000/auth/callback`
 - `https://YOUR-APP.onrender.com/auth/callback`
8. Click **Create**. You get a **Client ID** (ends in `.apps.googleusercontent.com`) and a **Client secret**. Copy both. The console shows only the last four characters of the secret afterwards.

The client ID is not sensitive and ends up in your app anyway. The client secret is a real secret and belongs in an environment variable, same as every other token in this guide.

Google login alone does not protect anything. If your app accepts every Google account that signs in, you have built a door that opens for four billion people. Signing in proves identity; it does not grant permission. The app has to check the returned email against a list you control. Project 3 does this with an `ALLOWED_EMAILS` variable, and that list is the actual security boundary.

3.5 Canvas access token

A Canvas token lets a program read and write your Canvas courses through the official Canvas API. It carries **your full Canvas permissions**, which for an instructor includes student names, submissions, and grades. That is FERPA-protected data. Three rules before you generate one:

- **Check your institution's policy** on student data in external tools, AI tools included. An institutional Claude for Education account may be covered by a data agreement; a personal account is not. When in doubt, ask, or practice on a sandbox course with no real students. Instructure offers free "Free-for-Teacher" Canvas accounts for exactly this kind of practice.
- **Set an expiration date** on the token.

- **Delete the token when the task is done.** Regenerating takes one minute.

Steps:

1. Log into your institution's Canvas in a browser.
2. In the left global navigation, click **Account** (your profile picture), then **Settings**.
3. Scroll down to **Approved Integrations** and click **+ New Access Token**.
4. **Purpose:** name the task, e.g. `claude-announcement-drafts`. Set an **expiration date**.
5. Click **Generate Token** and copy it once.

If you do not see an Approved Integrations section or the button is missing, your institution has restricted self-service tokens (several universities now require a request process instead). Contact your LMS support team.

You will also need your **Canvas base URL**: the address in your browser when you use Canvas, usually `https://yourschool.instructure.com` or a custom domain like `https://canvas.uw.edu`.

3.6 Resend API key (optional, for sending email)

Skip this unless you want an app that sends email. Two reasons you might: your app needs to notify someone (a weekly digest, a nudge to a student, an alert to yourself), or you want magic-link logins for people who cannot use the Google button from 3.4.

Sending email from a program is one of those problems that looks trivial and is not. Mail servers are suspicious of new senders, and a hand-rolled sender lands in spam or nowhere. Resend does the unglamorous part.

1. Go to `resend.com` (<https://resend.com>) and sign up. Google and GitHub sign-in both work.
2. Open **API Keys** and click **Create API Key**. Name it for the app. Give it **Sending access** only, not full access.

3. Copy the key (it starts with `re_`). Shown once, same as every other token here.

You can send immediately from `onboarding@resend.dev` without touching DNS, which is enough to get an app working. Mail from that address is for testing: it looks like what it is, and recipients' spam filters agree. To send from your own address, add a domain under **Domains** and paste the SPF and DKIM records Resend gives you into your DNS. If your address is `@uw.edu`, you do not control that DNS and central IT will not add records for your side project, so use a domain you own or stay on the test address.

The free tier as of August 2026 is 3,000 emails a month, capped at **100 a day**, on one domain. The daily cap is the one that bites: it is plenty for a dashboard emailing you, and not plenty for a mailing to 150 students. When you hit the cap on the free plan, sending pauses until the window rolls over. Nothing bounces, nothing bills you, the mail just does not go.

Before you email students from an app you built: that is a message from you, in your professional capacity, sent by a program you have not tested much. Send to yourself first. Every time.

3.7 Supabase project (optional, for storing data)

Skip this unless your app needs to remember something after the browser closes. Project 1 and Project 2 as written do not: the course site is files, and the quiz app holds everything in the page. The moment you want saved progress, submitted responses, or a record that accumulates over a term, you need a database, and Supabase is a Postgres database with a web dashboard and an API in front of it.

1. Go to supabase.com (<https://supabase.com>) and sign up. GitHub and Google sign-in both work.
2. Create an organization (Free plan), then **New project**.
3. Name it, set a **database password** (save it in your password manager now, it is shown once), and pick the region closest to your students. `West US` for the Pacific Northwest.
4. Wait a couple of minutes while it provisions.

5. Open **Settings**, then **API**. You need the **Project URL** and two keys.

The two keys are not interchangeable and mixing them up is the classic Supabase mistake:

- The **anon** key (also called publishable) is designed to sit in front-end code where anyone can read it. That is fine by design, and only by design, as explained below.
- The **service_role** key (also called secret) bypasses every access rule you write. Server-side only, environment variable only. If it ends up in a page, a repo, or a chat you later share, rotate it immediately.

Row Level Security is the part that actually protects the data.

A fresh table is open: with RLS off, the anon key lets anyone who views your page read and write every row in it. Supabase warns you about this in the dashboard and people click past it. Tell Claude to enable RLS on every table and write explicit policies, then check it yourself under **Authentication**, then **Policies**. A table with no policy and RLS off is a public spreadsheet with extra steps.

Free tier as of August 2026: 500 MB of database, two active projects, no backups. Two things follow from that.

Free projects pause after about a week without database activity. This is the one that ruins a teaching tool. You build a quiz app in October, students use it before the December final, and by then the project has been asleep for six weeks and the app returns errors. Supabase emails a warning first and restoring takes a click and roughly thirty seconds, but you will not be watching, and the students hit it at 11pm the night before. If an app has to be up on a date you care about, resume it a few days ahead and check it yourself, or ask Claude for a scheduled GitHub Action that pings the database every few days.

No backups on the free plan. Nothing is snapshotted for you. If the data matters, ask Claude for a script that exports the tables to CSV, and run it now and then.

On student data: a Supabase project is one more third party holding whatever you put in it. The privacy questions in 3.5 apply here with more force, because a database persists by definition. Anonymous or aggregate data is a different conversation from named submissions, and named submissions deserve a word with your privacy office first.

Claude can also talk to Supabase through a connector, the same way it does Netlify and Render (see Part 4). Enable it from the connectors menu if you would rather not paste keys into chats.

Part 4: Connect Claude to your accounts

There are two ways Claude reaches your accounts, and this guide uses both.

Connectors are set up once in Claude's settings. You sign in through your browser (OAuth), no token changes hands, and you can revoke access with one click. Use these when they exist: Netlify and Render both have one.

Pasting a token in chat works for everything else. You hand Claude a token in your message, Claude uses its built-in code environment to call the service's API, done. It is cruder but universal, and you control the blast radius by controlling the token. GitHub and Canvas use this route.

One caveat about the paste route: the token sits in that conversation's history afterwards. Delete the conversation when the work is done, or rotate the token. Pick one and actually do it.

4.1 Where connectors live

- Open claude.ai/customize/connectors (also reachable via **Settings > Connectors**).
- Inside a chat, connectors are toggled per conversation: click the + (or sliders) button at the lower left of the message box, then **Connectors**.

The Free plan includes directory connectors plus **one** custom connector, which is exactly enough for this guide.

4.2 Netlify (official connector)

1. On the Connectors page, find **Netlify** in the connector directory and click **Connect**.
2. A browser window opens; sign into Netlify and click **Authorize**.
3. That is the whole setup. No token.

If Netlify is not listed in your directory, add it as a custom connector with the URL `https://netlify-mcp.netlify.app/mcp`.

Test it in a new chat: *"Using the Netlify connector, list my Netlify sites."* An empty list is the correct answer right now.

4.3 Render (custom connector)

Render's connector is added by URL. Per Render's docs (<https://render.com/docs/mcp-server>):

1. On the Connectors page, click **Add custom connector**.
2. **Name:** `render`. **URL:** `https://mcp.render.com/mcp`.
3. Open **Advanced settings** and set **OAuth Client ID** to `claude`.
4. Click **Add**, then **Connect**, and approve in the browser window that opens. No API key needed for this route.

First prompt in any chat that uses it: *"Set my Render workspace to [YOUR WORKSPACE NAME]"* (your workspace name is at the top left of the Render dashboard). Then try *"List my Render services."*

Prefer not to use a connector? Paste your API key from 3.3 into the chat instead and Claude will call the Render API directly.

4.4 GitHub (paste the token)

No connector needed. In any chat, hand Claude the classic token from 3.1 along with the task:

Here is my GitHub token: ghp_XXXXXXX

Create a public repo called `test-drive` under my account, add a README that says hello, and send me the link. Do not put the token in any file.

Claude uses its code environment to run `git` and call the GitHub API with your token. That last line about not putting the token in a file is belt-and-braces. Say it anyway.

4.5 Canvas (paste the token)

Same pattern, plus the base URL:

My Canvas base URL is `https://yourschool.instructure.com`
(`https://yourschool.instructure.com`) and here is my Canvas token: XXXX

List my active courses with their course IDs. Read-only for now; do not change anything.

Start read-only. Once you trust the setup, you can allow writes (posting an announcement, creating an assignment) one action at a time. And per 3.5: sandbox course first, delete the token after.

Part 5: Token hygiene

Seven rules. All seven together take less time than cleaning up one compromised account.

1. **One token per purpose**, named for the job. Never reuse a token across unrelated projects.
2. **Always set an expiration**. 90 days on GitHub, explicit dates on Netlify, Render (where offered), and Canvas.
3. **Tokens never go in files**. Not in code, not in a README, not anywhere in a repo. Tell Claude this explicitly every time it builds

something. For deployed apps, secrets belong in the hosting service's environment variables (Project 3 shows how).

4. **Delete or rotate after a heavy session.** Especially any session where a token was pasted into chat. Rotation costs two minutes.
 5. **If a token leaks**, revoke it first and investigate second. Every service above has a revoke/delete button next to the token list.
 6. **An OAuth client secret is a token too.** Environment variable, never the repo. If it leaks, rotate it in the Google console under **Clients**.
 7. **Canvas tokens outrank the other rules.** They reach student data. Your institution's policy beats anything in this guide.
-

Part 6: Three projects

Each project has a starter prompt. Paste it into a fresh chat, fill in the CAPITALIZED bits, change whatever you like. Expect some back and forth before it is right. That is how this works.

Every starter prompt ends with "Ask me any questions before you start." Keep that line. I think it is the most useful sentence in this guide. Claude guesses when it does not know, and the guesses are plausible and wrong in the ways that eat your afternoon: invented office hours, a 10-week schedule, some framework you never asked for. Told to check first, it comes back with three questions and then builds the thing you meant.

It runs the other way too. When something confuses you, ask. "Why React here?" "What does deploy actually mean?" "Explain that command, I do not recognize it." You cannot wear out its patience, and nobody else is watching. Treat it like a colleague who happens to have read all the documentation.

Project 1: Course website (30 to 60 minutes)

What you get. A public course site: description, schedule, office hours, policies. Version-controlled on GitHub, live on Netlify, updated by asking Claude for edits.

You need. GitHub token (3.1), Netlify connector (4.2). Enable the Netlify connector in the chat.

Start from last year's syllabus. This is the shortcut. Drag your existing syllabus into the chat (Word, PDF, or a Canvas page pasted in as text) and let Claude pull the structure out of it: description, objectives, grading breakdown, policies, weekly topics. Correcting a draft is much faster than dictating one, and the result sounds like your course because it came from your course.

No syllabus handy? The second prompt starts from a blank page. Both end up in the same place.

With a syllabus to upload:

```
Here is my GitHub token: PASTE_GITHUB_TOKEN

Attached is my syllabus from last year. Build a course website from it
for COURSE NUMBER: COURSE TITLE, Autumn 2026.

Pull the course description, objectives, grading breakdown, and policies
from the syllabus. Flag anything that looks out of date (old dates, old
textbook editions, dead links) instead of copying it forward silently.
Rebuild the schedule against the Autumn 2026 calendar below.

Clean and readable, works on phones, no frameworks needed. Create a
public GitHub repo called COURSE-NUMBER-website under my account and
push everything. Do not put the token in any file.

Then deploy to Netlify using the Netlify connector and give me the URL.

Ask me any questions before you start.
```

Starting from scratch:

```
Here is my GitHub token: PASTE_GITHUB_TOKEN

Create a public GitHub repo called COURSE-NUMBER-website under my account.
Build a course website for COURSE NUMBER: COURSE TITLE, Autumn 2026.
Pages: home (course description, instructor, office hours: YOUR HOURS),
a week-by-week schedule built on the Autumn 2026 calendar below, and a
policies page (late work, collaboration, AI use). Clean and readable,
works on phones, no frameworks needed. Push everything to the repo. Do
not put the token in any file.

Then deploy the site to Netlify using the Netlify connector and give me
the live URL.

Ask me any questions before you start.
```

Paste this calendar block into either prompt (University of Washington, Autumn 2026):

```
UW Autumn 2026 quarter calendar:
- Instruction: September 30 (Wednesday) through December 11 (Friday)
- Length: 11 weeks. Note that Autumn is the long quarter at UW;
  Winter and Spring are 10 weeks.
- Week 1 is a partial week: Wednesday through Friday.
- Finals: December 12 through December 18
- No classes: Veterans Day, November 11 (Wednesday)
- No classes: Thanksgiving, November 26 (Thursday)
- No classes: Native American Heritage Day, November 27 (Friday)
```

```
Build the schedule around these dates. Do not schedule content or due
dates on the closure days, and do not silently shift material into
finals week.
```

Not at UW? Swap in your own calendar in the same shape. Spell out the length either way. The eleven-week Autumn quarter is a UW oddity, and left to itself Claude assumes a fifteen-week semester and hands you a schedule with four weeks that do not exist.

Then try. "Change my office hours to Wednesdays 1-3pm and redeploy." "Add a resources page with these five links." "The Thanksgiving week only has two class days, rebalance the topics." Each edit lands in the repo, so you have the full history.

Stretch. Point a custom domain at it, or ask Claude to add your publications page.

Project 2: Practice-problem app (1 to 2 hours)

What you get. An interactive quiz for your subject. Students pick answers, get instant feedback and explanations, see a score. No accounts, no server, no student data collected: it is a static page, which keeps both the engineering and the privacy story simple.

You need. Same as Project 1: GitHub token, Netlify connector.

Starter prompt:

Here is my GitHub token: PASTE_GITHUB_TOKEN

Create a public GitHub repo called SUBJECT-practice under my account. Build a single-page practice quiz app for TOPIC (e.g. "equilibrium problems in first-year chemistry"). Requirements:

- 10 multiple-choice questions with 4 options each. Write the questions at the level of AUDIENCE (e.g. "second-year undergraduates"). I will review and correct them, so make them your best attempt.
- After each answer: immediately show right/wrong plus a 2-3 sentence explanation.
- A running score and a final summary screen with a "try again" that reshuffles question order.
- Works well on phones. Store nothing about the user. No token in any file.

Push to the repo, deploy to Netlify with the Netlify connector, give me the URL.

Ask me any questions before you start.

Then try. Paste in your own question bank ("replace the questions with these 20"). Upload an old problem set or exam and ask Claude to build the quiz from those problems instead of inventing new ones. Ask for LaTeX-rendered math if your subject needs it. Ask for a topic filter if you add more questions.

Optional: save results with Supabase. As built, this app stores nothing, which is why it needs no privacy conversation. Adding a database changes that, so add it deliberately. Anonymous aggregates ("question 7 is missed by 80 percent of attempts") are useful for your teaching and carry almost none of the risk. Named per-student results are a different thing entirely, and I would not collect them without asking your privacy office first.

With a project set up per 3.7:

Add anonymous result tracking to the quiz app using Supabase.

Store one row per answered question: question id, whether it was correct, and a timestamp. No names, no emails, no student identifiers, no IP addresses, and no session id that could link one student's answers together across questions.

Enable Row Level Security on the table. The anon key may insert rows and may not read them back. Show me the policy you wrote and explain what it allows, because I want to check it myself.

Add a separate page for me that shows the percentage correct per question, sorted worst first.

Ask me any questions before you start.

Read the questions before you share the link. Claude writes plausible questions. You are the one who knows if they are right. Same review you would give a new TA's problem set, and it goes quickly. Fluency is not accuracy.

Project 3: Canvas assistant (an afternoon)

What you get. A web dashboard, hosted on Render, that talks to the Canvas API: it lists your courses, shows assignment submission counts and a grade distribution, and drafts announcement text you can paste into Canvas. Your Canvas token lives in a Render environment variable, never in the code.

You need. GitHub token (3.1), Render connector (4.3), Canvas token and base URL (3.5), Google OAuth client (3.4). Optionally a Resend key (3.6) if you add magic links in Step 4. Read the FERPA notes in 3.5 first. Start on a sandbox course with no real students.

Where this app lives. Free Render services are public web services. There is no private option on the free tier, so the URL is reachable by anyone on the internet who finds it, and `canvas-dashboard.onrender.com` is not hard to guess. Step 3 replaces the shared password with Google sign-in and an email allowlist, which is the difference between a locked door and a sign saying please do not enter.

If the app will hold real student data, the safer answer is not to deploy it at all. Ask Claude to run it on your own machine instead, reachable only at `localhost`. You give up being able to open it from your phone, and you gain a service that no stranger can reach because it is not on the internet. For a dashboard only you look at, that trade is usually the right one.

Step 1, explore in chat first (no app yet, just check the plumbing):

```
My Canvas base URL is https://YOURSCHOOL.instructure.com and here is my
Canvas token: PASTE_CANVAS_TOKEN
```

```
Read-only: list my active courses with IDs, and for course COURSE_ID
list the assignments with due dates and how many submissions each has.
```

Step 2, build and deploy:

```
Now build the dashboard. Here is my GitHub token: PASTE_GITHUB_TOKEN
```

```
Create a private GitHub repo called canvas-dashboard under my account.
Build a small Python (Flask) web app that reads two environment
variables, CANVAS_TOKEN and CANVAS_BASE_URL, and shows: my course list;
per course, assignments with due dates and submission counts; per
assignment, a simple grade distribution chart; and a button that drafts
an announcement summarizing upcoming deadlines (draft only, nothing is
posted to Canvas).
```

```
Leave authentication out for now, I will add Google login next. Do not
add a password login I will have to rip out.
```

```
Never write any token or password into the code or the repo.
```

```
Then use the Render connector to create a free web service from the
repo, set the environment variables (ask me for the values one at a
time), deploy, and give me the URL.
```

```
Ask me any questions before you start.
```

Step 3, put a real lock on it:

Now the dashboard is live and anyone who finds the URL can read it. Add Google sign-in. You need the client ID and secret from 3.4 and your app's URL from Step 2, which is also the hostname to put in the redirect URI back in the Google console.

Add Google sign-in to the dashboard.

Use the OAuth authorization code flow with these environment variables: `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `ALLOWED_EMAILS` (a comma-separated list), and `FLASK_SECRET_KEY` for session signing. Request only the `openid`, `email`, and `profile` scopes.

Rules:

- Every route except the login page and the OAuth callback requires a signed-in session. Do not leave any route open.
- After Google returns the profile, check the email against `ALLOWED_EMAILS`. Anyone not on the list gets a plain "not authorized" page and no data, even though Google authenticated them fine.
- Verify the `email_verified` claim too.
- The callback path must be `/auth/callback` so it matches what I registered with Google.
- Add a sign-out route that clears the session.
- Generate a random `FLASK_SECRET_KEY` for me to paste into Render. It must not be in the code.

Then redeploy to Render, set the new environment variables, and remind me which redirect URI to add in the Google console.

Ask me any questions before you start.

Test it in a private browser window. Sign in with your own account and you should reach the dashboard. Sign in with a different Google account and you should be refused. If the second test lets you in, the allowlist is not being checked and the app is still wide open, so stop and tell Claude exactly that.

Step 4, optional: magic links instead of Google.

Skip this if Google login works for everyone who needs in. It exists for the case where it does not: a co-instructor at another institution, a TA whose account is on a different provider, anyone the Google button turns away. A magic link is a one-time sign-in URL emailed to an address you have already approved. No password, no Google account.

You need a Resend API key from 3.6. Add magic links alongside Google sign-in, not instead of it.

Add magic-link login as a second option next to the existing Google sign-in. Use Resend for delivery, with `RESEND_API_KEY` and `MAIL_FROM` as environment variables.

Flow: the user enters an email, and if it is on `ALLOWED_EMAILS` they get a sign-in link. If it is not on the list, show the exact same "check your inbox" message and send nothing, so the page cannot be used to find out who has an account.

Token rules, please follow these exactly:

- Generate with `secrets.token_urlsafe(32)`. Never a counter, a timestamp, or anything derived from the email.
- Store only a SHA-256 hash of the token, never the token itself.
- Expire after 15 minutes.
- Single use: delete it the moment it is redeemed, and issue a fresh session cookie at that point.
- Rate limit to 3 requests per email address per 15 minutes.

Explain in your reply where the tokens are stored and what happens to them on restart, because I want to know if a free-tier restart logs everyone out.

Ask me any questions before you start.

Test the failure cases, not the happy path. Use a link twice (the second should be refused), request one for an address that is not on the allowlist (nothing should arrive, and the page should not say so), and let one sit for twenty minutes before clicking.

Worth saying plainly: a magic link in an inbox is a key to your dashboard. Anyone reading that inbox can use it. That is the tradeoff you accept for skipping passwords, and it is why the 15-minute expiry matters.

Then try. "Add a page that flags students with no submissions in the last two weeks." "Draft individual nudge emails I can review." Keep the whole thing in draft-and-review mode. The professor clicks send.

Free-tier reality. The free Render service sleeps when idle; the first load after a quiet spell takes a moment. Fine for a personal dashboard.

Part 7: Scheduled jobs with GitHub Actions

You already have this one. It needs no new account, no token, and no server, and it closes several gaps the projects leave open.

GitHub Actions runs a script on a schedule in GitHub's infrastructure. Public repositories get unlimited minutes; private ones get a monthly allowance on the Free plan that a few small cron jobs will not come close to using. Three things it is good for here:

- **Keeping a Supabase project awake.** A query every few days resets the inactivity clock from 3.7, so the quiz app still works in December.
- **A weekly digest.** Pull from Canvas, email yourself a summary through Resend (3.6). No always-on server, because the job runs for thirty seconds and stops.
- **Backups.** Export your Supabase tables to CSV on a schedule and commit them, which is the backup the free tier does not give you.

Secrets go in the repository, never in the workflow file: **Settings**, then **Secrets and variables**, then **Actions**, then **New repository secret**. The workflow reads them by name, and anyone reading your code sees the name and not the value.

```
Add a GitHub Actions workflow to this repo that runs every three days and
makes one small query against my Supabase project, so it does not get
paused for inactivity.
```

```
Read the credentials from repository secrets named SUPABASE_URL and
SUPABASE_ANON_KEY. Nothing sensitive in the workflow file. Add a
workflow_dispatch trigger too so I can run it by hand to test.
```

```
Tell me exactly which secrets to add and where, and what a successful run
looks like in the Actions tab.
```

```
Ask me any questions before you start.
```

Two things about scheduled workflows that catch people out. The times are approximate and can run late when GitHub is busy, so never schedule anything to the minute. And **GitHub disables scheduled workflows in a repository with no activity for 60 days**, emailing you first. On a repo you are actively working in this never comes up. On a keep-alive repo you deliberately ignore, it is

exactly the situation, and the job stops silently right when you have stopped thinking about it. Check the Actions tab at the start of each term.

Part 8: Appendix, Claude Code

Everything so far runs in the claude.ai chat. Claude Code is the same Claude working in a terminal on your own computer: it reads and edits files in a folder, runs programs, and uses git directly. Worth adopting when projects outgrow the chat window, when you want the files local, or when a session should pick up exactly where the last one stopped.

Two things up front. It needs a paid plan (Pro or higher, included in the subscription, so no per-use metering and no API key to look after). And it lives in a terminal, which sounds worse than it is. You type English at it, same as the chat.

Install

macOS or Linux, in Terminal:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows, in PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

Homebrew users: `brew install --cask claude-code`.

First session

```
mkdir my-first-project && cd my-first-project  
claude
```

On first run, pick **Claude.ai account** and approve in the browser. Then just talk to it: "Build the Project 2 quiz app in this folder." It asks permission before changing files or running commands. Read what it proposes before you say yes, at least until it has earned some trust.

Connect Netlify and Render

Claude Code uses the same connectors, added from the command line:

```
claude mcp add --transport http netlify https://netlify-mcp.netlify.app/mcp
claude mcp add --transport http --client-id claude render https://mcp.render.com/mcp
```

Then inside a session, type `/mcp` and authenticate each one in the browser. Your GitHub token gets pasted into the session the same way as in chat, or configured once with `git` credentials if you go further down this road.

Two habits

- Run `/init` in a project folder once: Claude Code writes a `CLAUDE.md` file describing the project, which future sessions read automatically. Standing instructions ("never commit tokens", "log changes in CHANGES.md") belong there.
- Prefer terminal-free? The Claude Code **VS Code extension** gives the same tool a graphical home.

Full documentation: code.claude.com/docs (<https://code.claude.com/docs>).

Troubleshooting

Claude says it cannot install a package, or a data-analysis step fails. Network egress is off. Turn it on in a browser: **Settings > Capabilities**, very bottom of the page (Part 1.3). The phone app does not show this setting.

Claude built something different from what you meant. You did not ask it to ask. Add "Ask me any questions before you start" and run it again. One extra exchange, and you skip the rebuild.

"Bad credentials" or 401 when Claude uses a token. The token expired, a scope is missing (GitHub needs `repo` and `workflow`), or the paste picked up a stray space. Generate a fresh one; it is faster than debugging.

Google sign-in fails with `redirect_uri_mismatch`. The URI registered in the Google console and the one your app sends have to match exactly: scheme, hostname, port, path, no trailing slash. Paste both into the chat and let Claude compare them character by character. This is the single most common OAuth error and it is always a typo.

Google says the app is unverified. Expected for an External app in testing mode. Click through the advanced link. It goes away if you switch the project to Internal, and it never applied to you in the first place if you stayed on `openid`, `email`, and `profile`.

A colleague you added to `ALLOWED_EMAILS` still cannot get in. On an External app they also need to be listed under **Test users** in the Google console. Two separate lists, both required.

A scheduled workflow stopped running. Look for a 60-day inactivity disable (there is a banner in the Actions tab and a button to re-enable). Otherwise check that the cron syntax is what you meant, since a wrong field silently means a schedule you did not intend.

An app that worked in October is broken in December. Most likely a paused Supabase project (about a week of inactivity does it) or a sleeping Render free service. Open the relevant dashboard and look at the project status before you debug any code.

Magic-link emails never arrive. Check the Resend dashboard logs first, which tell you whether the send failed or the mail was delivered and filtered. Mail from `onboarding@resend.dev` lands in spam routinely. Also check whether you hit the 100-a-day free cap, in which case sending is paused rather than broken.

Canvas has no "Approved Integrations" section. Your institution turned off self-service tokens. Ask LMS support. Several universities hand them out through a request form instead.

Claude ignores a connector. Enable it for that conversation: **+** button at the lower left of the message box, then **Connectors**, toggle it on. Also confirm it shows as connected on the Connectors settings page.

"Add custom connector" refuses or is missing. The Free plan allows one custom connector; remove an old one or upgrade.

Render app is slow on first load. Free services sleep when idle. It wakes on its own; wait a moment and reload.

You hit the usage limit mid-project. It resets on a rolling 5-hour window. Come back after the reset, or take it as your sign about Pro.

Everything worked yesterday and today it does not. Something upstream changed, or a token quietly expired. Paste the error into the chat and ask Claude what broke. It is better at reading its own stack traces than you would expect.

A token got committed to a repo by accident. Revoke it immediately (the service's token page has a delete button), then generate a new one. Revoking first makes the leaked copy worthless; cleaning git history is optional afterwards.

Official documentation

- Claude plans: claude.com/pricing (<https://claude.com/pricing>) · Connectors: support.claude.com (<https://support.claude.com/en/articles/11175166-get-started-with-custom-connectors-using-remote-mcp>)

- **GitHub accounts and tokens:** [docs.github.com](https://docs.github.com/en/get-started/start-your-journey/creating-an-account-on-github) (<https://docs.github.com/en/get-started/start-your-journey/creating-an-account-on-github>) · github.com/settings/tokens (<https://github.com/settings/tokens>)
 - **Netlify:** [docs.netlify.com](https://docs.netlify.com/api-and-cli-guides/api-guides/get-started-with-api/) (<https://docs.netlify.com/api-and-cli-guides/api-guides/get-started-with-api/>) and the **Netlify + Claude** page (<https://www.netlify.com/with/claude/>)
 - **Render:** render.com/docs/mcp-server (<https://render.com/docs/mcp-server>) · render.com/docs/free (<https://render.com/docs/free>)
 - **Canvas API and tokens:** [Canvas OAuth docs](https://www.canvas.instructure.com/doc/api/file.oauth.html) (<https://www.canvas.instructure.com/doc/api/file.oauth.html>) · **managing access tokens** (<https://community.canvaslms.com/t5/Canvas-Basics-Guide/How-do-I-manage-API-access-tokens-in-my-user-account/ta-p/615312>)
 - **Claude Code:** code.claude.com/docs (<https://code.claude.com/docs>)
-

Version 21 Aug 2026. Maintained by Yusuf Pisan (<https://github.com/pisanuw>), pisan@uw.edu. Corrections and questions: email me or open an issue.

Written for colleagues who suspect this stuff might be useful but have not had a free afternoon to find out. Now you have a map for the afternoon. Let me know how it goes. :-)